

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 6 DOUBLE LINKED LIST



Disusun Oleh :

DIKA GIOWANDA

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, MEI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat, hidayah, dan karunia-Nya sehingga laporan praktikum Struktur Data tentang implementasi *Double Linked List* menggunakan bahasa pemrograman Java ini dapat diselesaikan dengan baik dan tepat waktu. Sholawat serta salam senantiasa tercurah kepada Nabi Muhammad SAW, keluarga, sahabat, dan seluruh pengikutnya hingga akhir zaman.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban, refleksi, serta dokumentasi atas kegiatan praktikum yang telah dilaksanakan, dengan fokus pada penerapan konsep struktur data *Double Linked List*. *Double Linked List* merupakan salah satu struktur data linier di mana setiap elemen (*node*) memiliki dua tautan, yaitu penunjuk ke elemen berikutnya (*next*) dan penunjuk ke elemen sebelumnya (*previous*). Dalam praktikum ini, penulis mempelajari penggunaan bahasa Java untuk membuat kelas *Node*, memanipulasi *pointer*, serta mengimplementasikan operasi dasar seperti penyisipan data (*insert*), penghapusan data (*delete*), pencarian (*search*), dan penelusuran (*traversal*) baik secara maju maupun mundur.

Penyusunan laporan ini tidak mungkin terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang tulus kepada dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang berharga selama proses praktikum, serta kepada rekan-rekan mahasiswa yang turut berkontribusi dalam diskusi dan kolaborasi pemrograman.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami logika dan implementasi *Double Linked List* dalam bahasa Java.

Akhir kata, penulis berharap semoga ilmu yang diperoleh melalui praktikum ini dapat bermanfaat dan diterapkan dengan baik dalam pengembangan perangkat lunak selanjutnya.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
1.5 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Pemrograman Double Linked List.....	3
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 6.....	4
BAB 3 PENUTUP.....	18
3.1 Kesimpulan.....	18
3.2 Saran.....	18
DAFTAR PUSTAKA.....	19
LAPORAN TUGAS.....	20

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep pemrograman Java. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan penyusunan kode program, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar pemrograman Struktur Data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Java.
3. Memahami tipe data dasar hingga kompleks, termasuk konsep penyimpanan data.
4. Mampu menerapkan struktur data linear (*Linked List, Stack, Queue*) dan dalam bahasa pemrograman
5. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
6. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
7. Mengetahui dan mengaplikasikan *ADT, ArrayList, Stack, Queue, Single Linked List, Double Linked List, Sorting 1 (Insertion, selection, bubble), Sorting 2 (shell, quick, merge), dan Searching (BFS dan DFS)*.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang *Java Development Kit (JDK)* dan *Integrated Development Environment (IDE)* atau *Eclips* yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat Praktikum

Manfaat praktikum Struktur Data meliputi kemampuan memahami cara menyimpan data secara terstruktur, rapi, dan efisien, sehingga mengurangi penggunaan memori dan mempercepat akses data. Meningkatkan efisiensi penyimpanan dan manipulasi data, mengoptimalkan kecepatan algoritma, serta mempermudah penyelesaian masalah pemrograman yang kompleks. Selain itu, praktikum ini juga membantu membangun fondasi untuk mengembangkan aplikasi lintas platform yang lebih kompleks di berbagai bidang teknologi.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Pemrograman Double Linked List

Double Linked List (DLL) adalah struktur data linear yang terdiri dari kumpulan *node* terhubung, di mana setiap *node* memiliki dua *pointer*, yaitu satu ke *node* berikutnya (*next*) dan satu ke *node* sebelumnya (*prev*). Keunggulan utamanya dibandingkan *Single Linked List* adalah kemampuan penelusuran data dua arah (maju dan mundur).

Komponen Utama dalam Java:

- Node: Terdiri dari data, *pointer next*, dan *pointer prev*.
- Head: *Pointer* yang menunjuk ke *node* pertama. Jika kosong, *head* bernilai *null*.
- Tail: *Pointer* yang menunjuk ke *node* terakhir. Ini memudahkan operasi di akhir list.

Operasi Utama Praktikum:

- Insert/Add: Menambah data di awal (*addFirst*), akhir (*addLast*), atau posisi tertentu.
- Delete/Remove: Menghapus data di awal, akhir, atau posisi tertentu.
- Display/Traversal: Menampilkan data dari depan ke belakang (*next*) atau belakang ke depan (*prev*).

Berikut adalah komponen detailnya:

- Node (Simpul): Unit dasar yang menyimpan elemen data dan memiliki dua *pointer/referensi*.
- data: Nilai yang disimpan.
- next: Referensi ke *node* setelahnya.
- prev: Referensi ke *node* sebelumnya.
- Head (Head Pointer): Referensi ke *node* pertama dalam list.
- Tail (Tail Pointer): Referensi ke *node* terakhir dalam list.

- Null Pointer: Penanda akhir list, di mana next dari *tail* atau *prev* dari *head* menunjuk ke *null* (kecuali pada circular DLL).

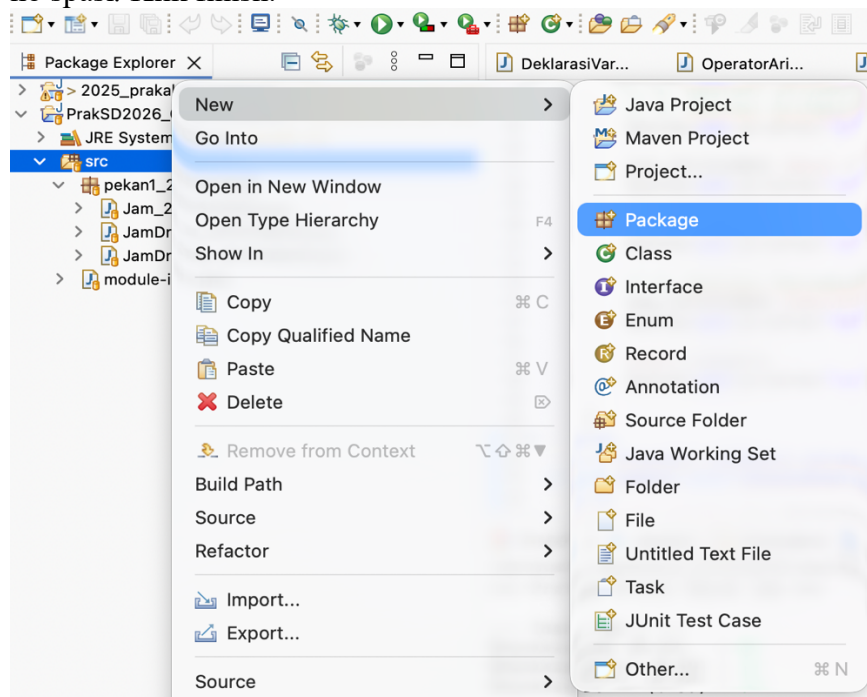
Operasi Utama DLL dalam Java:

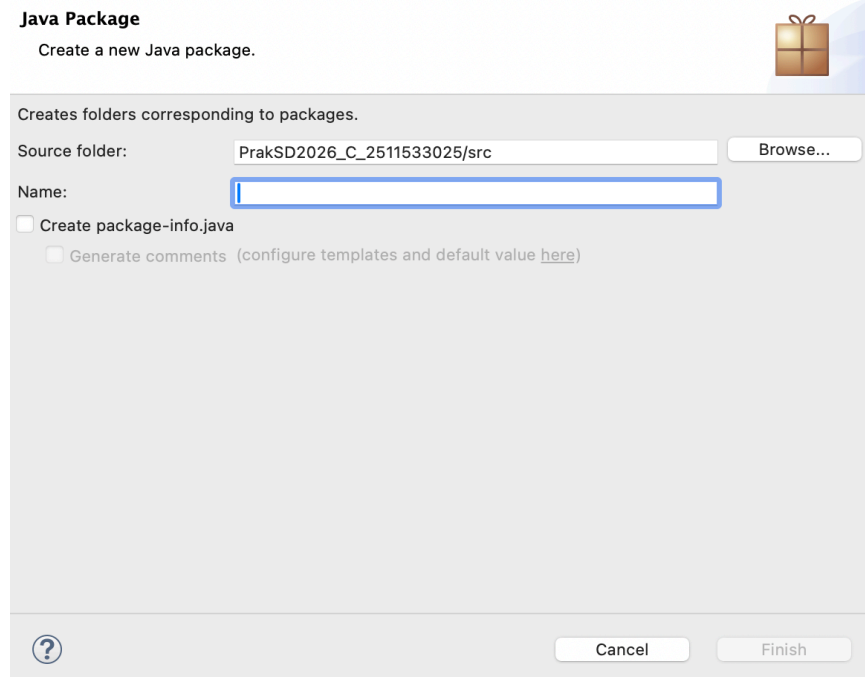
- `addFirst()` / `addLast()`: Menambahkan elemen di awal/akhir.
- `deleteFirst()` / `deleteLast()`: Menghapus elemen dari awal/akhir.
- `display()`: Menampilkan isi list dari depan maupun belakang.

2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 6

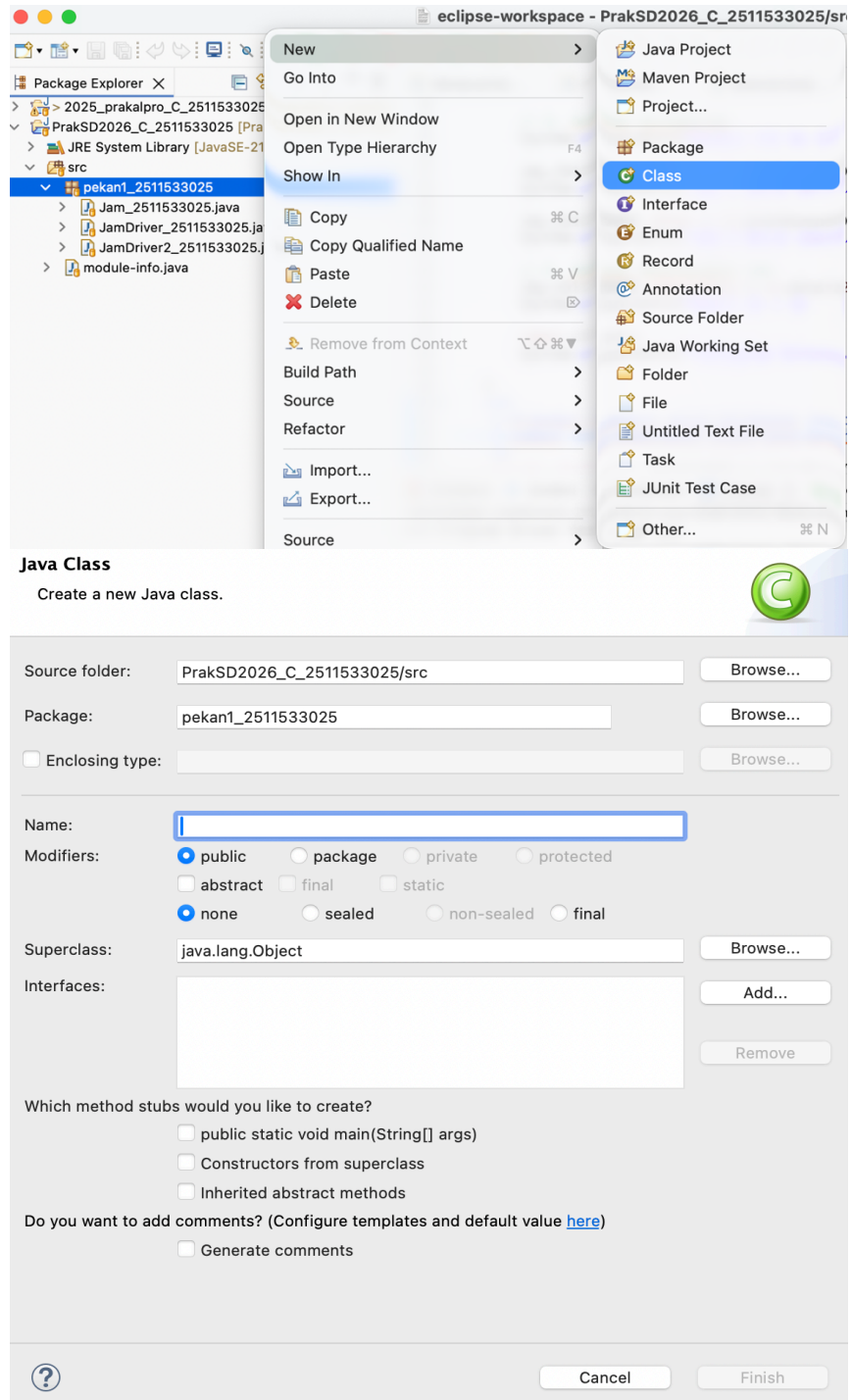
Sebelum ke langkah-langkah pengerjaan contoh proyek, saya akan menjelaskan terlebih dahulu gimana caranya menambahkan proyek nya terlebih dahulu :

1. Klik kanan pada folder `src`, pilih `new`, pilih `package`. Nanti akan muncul halaman Java Package, cukup masukan nama, nama wajib no spasi. Klik finish.





2. Klik kanan pada folder package, pilih new, pilih class. Nanti akan muncul halaman Java Class, cukup masukan nama class sesuai projek yang akan dibuat, nama wajib no spasi. Klik finish.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru.

- A. Baiklah karena proses menambahkan proyek sudah dijelaskan, selanjutnya langkah-langkah pengerjaan proyek pekan 6:

a. Class NodeDLL_2511533025

1. Kode dari tiga komponen utama: variabel data_3025 bertipe integer yang digunakan untuk menyimpan nilai atau informasi pada simpul tersebut, serta dua variabel referensi objek yaitu next_3025 yang berperan sebagai penunjuk (pointer) ke simpul berikutnya dan prev_3025 sebagai penunjuk ke simpul sebelumnya.

```
1 package Pekan6_2511533025;
2
3 public class NodeDLL_2511533025 {
4     // mendefinisikan kelas Node
5     int data_3025; // data
6     NodeDLL_2511533025 next_3025; // Pointer ke next node
7     NodeDLL_2511533025 prev_3025; // Pointer ke previous node
8 }
```

2. Kode ini untuk menginisialisasi objek node (simpul) baru dalam struktur data Doubly Linked List.

```
9     // konstruktor
10 public NodeDLL_2511533025(int data) {
11     this.data_3025 = data;
12     this.next_3025 = null;
13     this.prev_3025 = null;
14 }
15
16 }
17 }
```

b. Class InsertDLL_2511533025

Class ini merupakan sambungan dari class NodeDLL agar kode dapat berjalan yaitu mengeluarkan output.

1. Kode dari metode insertBegin ini berfungsi untuk menambahkan sebuah simpul baru di posisi paling depan (awal) pada struktur data Doubly Linked List. Secara teknis, fungsi ini pertama-tama membuat objek simpul baru (new_node_3025) dengan data yang diberikan, kemudian menghubungkan penunjuk next simpul baru tersebut ke simpul head yang lama agar ia menjadi elemen pertama. Jika daftar tersebut tidak kosong (head_3025 != null), fungsi ini akan memperbarui penunjuk prev dari simpul head yang lama untuk menunjuk kembali ke simpul baru, sehingga hubungan dua arah tetap terjaga. Terakhir, fungsi mengembalikan simpul baru tersebut sebagai head yang baru untuk daftar tersebut.

```

1 package Pekan6_2511533025;
2
3 public class InsertDLL_2511533025 {
4     // menambah node di awal DLL
5     static NodeDLL_2511533025 insertBegin(NodeDLL_2511533025 head_3025, int data_3025) {
6         // buat node baru
7         NodeDLL_2511533025 new_node_3025 = new NodeDLL_2511533025(data_3025);
8         // jadikan pointer nextnya head
9         new_node_3025.next_3025 = head_3025;
10        // jadikan pointer prev head ke new_node
11        if (head_3025 != null) {
12            head_3025.prev_3025 = new_node_3025;
13        }
14        return new_node_3025;
15    }
}

```

2. Kode fungsi insertEnd ini bertujuan untuk menambahkan sebuah simpul baru berisi data integer (newData_3025) ke posisi paling akhir pada struktur data Doubly Linked List. Prosesnya dimulai dengan membuat objek simpul baru (newNode_3025), kemudian melakukan pengecekan apakah list masih kosong (head bernilai null); jika kosong, simpul baru tersebut langsung ditetapkan sebagai head. Jika list sudah berisi data, kode akan melakukan penelusuran (traversal) menggunakan perulangan while dari head hingga mencapai simpul terakhir, lalu menghubungkan simpul terakhir tersebut ke simpul baru melalui pointer next dan mengaitkan pointer prev pada simpul baru kembali ke simpul sebelumnya untuk menjaga integritas hubungan dua arah khas doubly linked list, sebelum akhirnya mengembalikan pointer head yang telah diperbarui.

```

16 // fungsi menambahkan node di akhir
17 public static NodeDLL_2511533025 insertEnd(NodeDLL_2511533025 head_3025, int newData_3025) {
18     // buat node baru
19     NodeDLL_2511533025 newNode_3025 = new NodeDLL_2511533025(newData_3025);
20     // jika dll null jadikan head
21     if (head_3025 == null) {
22         head_3025 = newNode_3025;
23     }
24     else {
25         NodeDLL_2511533025 curr_3025 = head_3025;
26         while (curr_3025.next_3025 != null) {
27             curr_3025 = curr_3025.next_3025;
28         }
29         curr_3025.next_3025 = newNode_3025;
30         newNode_3025.prev_3025 = curr_3025;
31     }
32     return head_3025;
33 }

```

3. Kode ini merupakan fungsi Java bernama insertAtPosition yang dirancang untuk menyisipkan sebuah simpul (node) baru ke dalam struktur data Doubly Linked List pada posisi tertentu. Di dalam cuplikan ini, fungsi menangani kasus khusus jika data ingin dimasukkan ke posisi pertama (pos_3025 == 1): pertama-tama program membuat objek simpul baru dengan data yang diberikan, kemudian mengatur penunjuk next dari simpul baru tersebut ke arah head yang lama, dan jika daftar tidak kosong, penunjuk prev dari head lama diarahkan balik ke simpul baru untuk menjaga hubungan dua arah. Terakhir, fungsi memperbarui head agar menunjuk ke simpul baru tersebut sebagai elemen pertama yang baru.


```

34 //fungsi menambahkan node di posisi tertentu
35 public static NodeDLL_2511533025 insertAtPosition(NodeDLL_2511533025 head_3025, int pos_3025, int new_data_3025) {
36     // buat node baru
37     NodeDLL_2511533025 new_node_3025 = new NodeDLL_2511533025(new_data_3025);
38     if (pos_3025 == 1) {
39         new_node_3025.next_3025 = head_3025;
40         if (head_3025 != null) {
41             head_3025.prev_3025 = new_node_3025;
42         }
43         head_3025 = new_node_3025;
44         return head_3025;
45     }

```

4. Kode ini untuk menyisipkan sebuah simpul baru (new_node_3025) ke dalam struktur data Doubly Linked List pada posisi tertentu yang ditentukan oleh variabel pos_3025. Algoritma dimulai dengan melakukan iterasi menggunakan looping untuk mencari simpul tepat sebelum posisi penyisipan; jika posisi tersebut tidak ditemukan atau melebihi panjang list (curr_3025 == null), program akan mencetak pesan kesalahan dan mengembalikan head asli. Jika posisi valid, kode akan mengatur empat referensi tautan secara berurutan: menghubungkan simpul baru ke simpul sebelumnya dan sesudahnya, mengarahkan simpul saat ini ke simpul baru, serta memperbarui referensi prev pada simpul setelahnya jika simpul tersebut bukan merupakan akhir dari list (null)..

```

46 NodeDLL_2511533025 curr_3025 = head_3025;
47 for (int i = 1; i < pos_3025 - 1 && curr_3025 != null; ++i) {
48     curr_3025 = curr_3025.next_3025;
49 }
50 if (curr_3025 == null) {
51     System.out.println("Posisi tidak ada");
52     return head_3025;
53 }
54 new_node_3025.prev_3025 = curr_3025;
55 new_node_3025.next_3025 = curr_3025.next_3025;
56 curr_3025.next_3025 = new_node_3025;
57 if (new_node_3025.next_3025 != null) {
58     new_node_3025.next_3025.prev_3025 = new_node_3025;
59 }
60 return head_3025;
61 }

```

5. Kode tersebut mendefinisikan sebuah metode statis bernama printList yang berfungsi untuk menampilkan seluruh elemen dari sebuah Doubly Linked List ke layar secara berurutan. Proses dimulai dengan menginisialisasi variabel penunjuk curr_3025 pada posisi kepala daftar (head), kemudian melakukan iterasi menggunakan perulangan while yang terus berjalan selama node saat ini tidak bernilai null. Di dalam setiap iterasi, program mencetak data dari node tersebut diikuti dengan simbol panah (<->) sebagai representasi visual hubungan antar node, lalu memindahkan penunjuk ke node berikutnya melalui referensi next_3025.

```

63 public static void printList(NodeDLL_2511533025 head_3025) {
64     NodeDLL_2511533025 curr_3025 = head_3025;
65     while (curr_3025 != null) {
66         System.out.print(curr_3025.data_3025 + " <-> ");
67         curr_3025 = curr_3025.next_3025;
68     }
69     System.out.println();
70 }

```


6. Kode tersebut adalah metode main yang mendemonstrasikan operasi dasar pada struktur data Doubly Linked List (DLL), dimulai dengan inisialisasi manual tiga simpul yang membentuk urutan 2 <-> 3 <-> 5. Selanjutnya, program melakukan serangkaian manipulasi list dengan memanggil fungsi insertBegin untuk menambahkan nilai 1 di awal, insertEnd untuk menambahkan nilai 6 di akhir, serta insertAtPosition untuk menyisipkan nilai 4 pada posisi keempat. Di setiap tahap pembaruan, fungsi printList dipanggil untuk mencetak isi list ke layar, sehingga hasil akhirnya menunjukkan urutan simpul yang lengkap dan terurut dari 1 hingga 6.

```
71 public static void main(String[] args) {  
72     // membuat dll 2 <-> 3 <-> 5  
73     NodeDLL_2511533025 head_3025 = new NodeDLL_2511533025(2);  
74     head_3025.next_3025 = new NodeDLL_2511533025(3);  
75     head_3025.next_3025.prev_3025 = head_3025;  
76     head_3025.next_3025.next_3025 = new NodeDLL_2511533025(5);  
77     head_3025.next_3025.next_3025.prev_3025 = head_3025.next_3025;  
78     // cetak DLL awal  
79     System.out.print("DLL Awal: ");  
80     printList(head_3025);  
81     // tambah 1 di awal  
82     head_3025 = insertBegin(head_3025, 1);  
83     System.out.print(  
84         "simpul 1 ditambah di awal: ");  
85     printList(head_3025);  
86     // tambah 6 di akhir  
87     System.out.print(  
88         "simpul 6 ditambah di akhir: ");  
89     int data_3025 = 6;  
90     head_3025 = insertEnd(head_3025, data_3025);  
91     printList(head_3025);  
92     // menambah node 4 di posisi 4  
93     System.out.print("tambah node 4 di posisi 4: ");  
94     int data2_3025 = 4;  
95     int pos_3025 = 4;  
96     head_3025 = insertAtPosition(head_3025, pos_3025, data2_3025);  
97     printList(head_3025);  
98 }  
99 }  
100
```

c. Class PenelusuranDLL_2511533025

Class ini merupakan sambungan dari class NodeDLL agar kode dapat berjalan yaitu mengeluarkan output.

1. Kode fungsi statis bernama forwardTraversal yang dirancang untuk melakukan penelusuran maju pada struktur data Doubly Linked List (DLL). Fungsi ini bekerja dengan menerima parameter head sebagai titik awal, kemudian menggunakan sebuah pointer bantuan curr_325 untuk menyusuri setiap simpul (node) di dalam list mulai dari depan hingga ujung akhir. Di dalam blok perulangan while, kode secara sistematis mencetak nilai data dari setiap simpul yang dikunjungi ke layar dengan format pemisah <->, lalu memindahkan penunjuk ke simpul berikutnya menggunakan properti .next_3025. Proses penelusuran ini akan terus berlanjut secara berurutan hingga mencapai nilai null.

```

1 package Pekan6_2511533025;
2 public class PenelusuranDLL_2511533025 {
3     //fungsi penelusuran maju
4     static void forwardTraversal(NodeDLL_2511533025 head_3025) {
5         // memulai penelusuran dari head
6         NodeDLL_2511533025 curr_3025 = head_3025;
7         // lanjutkan sampai akhir
8         while (curr_3025 != null) {
9             //print data
10            System.out.print(curr_3025.data_3025 + " <-> ");
11            //pindah ke node berikutnya
12            curr_3025 = curr_3025.next_3025;
13        }
14        //print spasi
15        System.out.println();
16    }

```

2. Kode fungsi bernama backwardTraversal yang bertujuan untuk menampilkan isi dari sebuah Doubly Linked List secara terbalik, dimulai dari elemen terakhir (ekor) hingga elemen pertama (kepala). Proses diawali dengan menerima parameter tail_3025 sebagai titik awal, lalu menggunakan perulangan while yang akan terus berjalan selama simpul saat ini (curr_3025) tidak bernilai null. Di dalam perulangan tersebut, program mencetak data dari simpul yang sedang diakses dan berpindah ke simpul sebelumnya menggunakan referensi prev_3025.

```

17 // fungsi penelusuran mundur
18 static void backwardTraversal(NodeDLL_2511533025 tail_3025) {
19     // mulai dari akhir
20     NodeDLL_2511533025 curr_3025 = tail_3025;
21     // lanjut sampai head
22     while (curr_3025 != null) {
23         //cetak data
24         System.out.print(curr_3025.data_3025 + " <-> ");
25         // pindah ke node sebelumnya
26         curr_3025 = curr_3025.prev_3025;
27     }
28     // cetak spasi
29     System.out.println();
30 }

```

3. Kode ini merupakan implementasi metode main untuk membangun dan menguji struktur data Doubly Linked List (DLL) sederhana yang terdiri dari tiga buah simpul. Program dimulai dengan menginisialisasi tiga objek node head_3025, second_3025, dan third_3025 dengan nilai masing-masing 1, 2, dan 3, kemudian menghubungkannya secara manual menggunakan pointer next dan prev untuk menciptakan hubungan dua arah antar simpul. Setelah struktur list terbentuk, kode tersebut memanggil fungsi forwardTraversal(head_3025) untuk mencetak isi list dari depan ke belakang dan fungsi backwardTraversal(third_3025) untuk melakukan penelusuran mundur.

```

31 public static void main(String[] args) {
32     // cetak DLL
33     NodeDLL_2511533025 head_3025 = new NodeDLL_2511533025(1);
34     NodeDLL_2511533025 second_3025 = new NodeDLL_2511533025(2);
35     NodeDLL_2511533025 third_3025 = new NodeDLL_2511533025(3);
36
37     head_3025.next_3025 = second_3025;
38     second_3025.prev_3025 = head_3025;
39     second_3025.next_3025 = third_3025;
40     third_3025.prev_3025 = second_3025;
41
42     System.out.println("Penelusuran maju:");
43     forwardTraversal(head_3025);
44
45     System.out.println("Penelusuran mundur:");
46     backwardTraversal(third_3025);
47 }
48 }

```

d. Class HapusDLL_2511533025

Class ini merupakan sambungan dari class NodeDLL agar kode dapat berjalan yaitu mengeluarkan output.

1. Kode fungsi bernama delHead yang bertujuan untuk menghapus simpul pertama (head) pada struktur data Doubly Linked List. Fungsi ini bekerja dengan cara memeriksa terlebih dahulu apakah daftar tersebut kosong; jika kosong, ia akan mengembalikan nilai null. Jika tidak kosong, fungsi akan memindahkan penunjuk simpul utama ke simpul berikutnya (head_3025.next_3025), lalu memutuskan hubungan simpul baru tersebut dengan simpul yang dihapus dengan mengatur penunjuk sebelumnya (prev_3025) menjadi null.

```

1 package Pekan6_2511533025;
2 public class HapusDLL_2511533025 {
3     // fungsi menghapus node awal
4     public static NodeDLL_2511533025 delHead(NodeDLL_2511533025 head_3025) {
5         if (head_3025 == null) {
6             return null;
7         }
8         NodeDLL_2511533025 temp = head_3025;
9         head_3025 = head_3025.next_3025;
10        if (head_3025 != null) {
11            head_3025.prev_3025 = null;
12        }
13        return head_3025;
14    }
15 }

```

2. Kode fungsi delLast yang dirancang untuk menghapus simpul (node) terakhir dalam sebuah struktur data Doubly Linked List. Fungsi ini bekerja dengan terlebih dahulu menangani kondisi khusus: jika daftar kosong (head bernilai null) atau hanya berisi satu elemen, maka fungsi akan mengembalikan null. Untuk daftar dengan lebih dari satu elemen, kode menggunakan perulangan while untuk menelusuri seluruh list hingga mencapai simpul terakhir (curr_3205). Setelah simpul terakhir ditemukan, hubungan antara simpul terakhir tersebut dengan simpul sebelumnya diputus dengan mengubah penunjuk next pada simpul sebelum terakhir menjadi null.

```

13 // fungsi menghapus di akhir
14 public static NodeDLL_2511533025 delLast(NodeDLL_2511533025 head_3025) {
15     if (head_3025 == null) {
16         return null;
17     }
18     if (head_3025.next_3025 == null) {
19         return null;
20     }
21     NodeDLL_2511533025 curr_3025 = head_3025;
22     while (curr_3025.next_3025 != null) {
23         curr_3025 = curr_3025.next_3025;
24     }
25     // update pointer previous node
26     if (curr_3025.prev_3025 != null) {
27         curr_3025.prev_3025.next_3025 = null;
28     }
29     return head_3025;
30 }

```

3. Kode mendefinisikan fungsi delPos yang berfungsi untuk menghapus sebuah node pada posisi tertentu (indeks pos_3025) dalam sebuah Doubly Linked List (DLL). Secara prosedural, fungsi ini pertama-tama memeriksa apakah list kosong; jika tidak, ia melakukan iterasi menggunakan bantuan pointer curr_3025 untuk mencari node pada posisi yang dituju. Setelah node ditemukan, fungsi melakukan pembaharuan pointer dengan menghubungkan node sebelum dan sesudah node yang akan dihapus (memutus rantai node tersebut), serta menyertakan pengecekan khusus jika node yang dihapus adalah node pertama (head) agar referensi head_3025 tetap akurat.

```

28 // fungsi menghapus node posisi tertentu
29 public static NodeDLL_2511533025 delPos(NodeDLL_2511533025 head_3025, int pos_3025) {
30     // jika DLL kosong
31     if (head_3025 == null) {
32         return head_3025;
33     }
34     NodeDLL_2511533025 curr_3025 = head_3025;
35     // telusuri sampai ke node yang akan dihapus
36     for (int i = 1; curr_3025 != null && i < pos_3025; ++i) {
37         curr_3025 = curr_3025.next_3025;
38     }
39     // jika posisi tidak ditemukan
40     if (curr_3025 == null) {
41         return head_3025;
42     }
43     // Update pointer
44     if (curr_3025.prev_3025 != null) {
45         curr_3025.prev_3025.next_3025 = curr_3025.next_3025;
46     }
47     if (curr_3025.next_3025 != null) {
48         curr_3025.next_3025.prev_3025 = curr_3025.prev_3025;
49     }
50     // jika yang dihapus head
51     if (head_3025 == curr_3025) {
52         head_3025 = curr_3025.next_3025;
53     }
54     return head_3025;
55 }

```

4. Kode fungsi statis bernama printList yang dirancang untuk menampilkan seluruh elemen dari sebuah Doubly Linked List (DLL) ke layar. Fungsi ini bekerja dengan menerima parameter berupa head (elemen pertama) dari list, kemudian menggunakan variabel bantuan curr_3025 untuk menelusuri setiap simpul mulai dari awal hingga akhir. Selama simpul saat ini tidak bernilai null, program akan mencetak data yang tersimpan di dalam simpul tersebut diikuti dengan simbol panah ganda (<->) sebagai representasi visual hubungan antar simpul dalam linked list. Setelah mencetak data, penunjuk curr_3025 akan berpindah ke simpul berikutnya melalui referensi .next_3025.

```

52 // fungsi mencetak DLL
53 public static void printList(NodeDLL_2511533025 head_3025) {
54     NodeDLL_2511533025 curr_3025 = head_3025;
55     while (curr_3025 != null) {
56         System.out.print(curr_3025.data_3025 + " <-> ");
57         curr_3025 = curr_3025.next_3025;
58     }
59     System.out.println();
60 }

```

5. Kode ini merupakan program utama (main method) dalam bahasa Java yang mensimulasikan manipulasi struktur data Doubly Linked List (DLL). Di awal, kode menginisialisasi lima buah node (dengan nilai 1 hingga 5) dan menghubungkannya secara manual menggunakan referensi .next dan .prev untuk membentuk urutan list yang saling terhubung dua arah. Program kemudian melakukan serangkaian operasi penghapusan, yaitu menghapus elemen pertama menggunakan fungsi delHead, menghapus elemen terakhir dengan delLast, serta menghapus elemen pada posisi spesifik (posisi ke-2) melalui fungsi delPos.

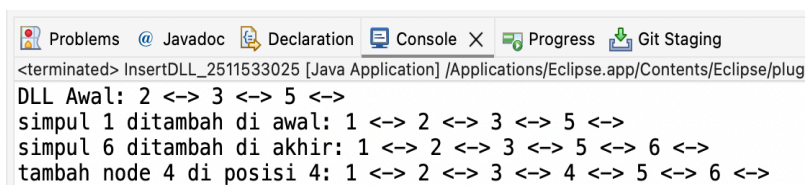
```

62 public static void main(String[] args) {
63     // buat sebuah DLL
64     NodeDLL_2511533025 head_3025 = new NodeDLL_2511533025(1);
65     head_3025.next_3025 = new NodeDLL_2511533025(2);
66     head_3025.next_3025.prev_3025 = head_3025;
67     head_3025.next_3025.next_3025 = new NodeDLL_2511533025(3);
68     head_3025.next_3025.next_3025.prev_3025 = head_3025.next_3025;
69     head_3025.next_3025.next_3025.next_3025 = new NodeDLL_2511533025(4);
70     head_3025.next_3025.next_3025.next_3025.prev_3025 = head_3025.next_3025.next_3025;
71     head_3025.next_3025.next_3025.next_3025.next_3025 = new NodeDLL_2511533025(5);
72     head_3025.next_3025.next_3025.next_3025.next_3025.prev_3025 = head_3025.next_3025.next_3025;
73
74     System.out.print("DLL Awal: ");
75     printList(head_3025);
76
77     System.out.print("Setelah head dihapus: ");
78     head_3025 = delHead(head_3025);
79     printList(head_3025);
80
81     System.out.print("Setelah node terakhir dihapus: ");
82     head_3025 = delLast(head_3025);
83     printList(head_3025);
84
85     System.out.print("menghapus node ke 2: ");
86     head_3025 = delPos(head_3025, 2);
87
88     printList(head_3025);
89 }
90 }

```

f. Hasil proyek pekan 6.

1. Output dari Class NodeDLL_2511533025 dan Class InsertDLL_2511533025.

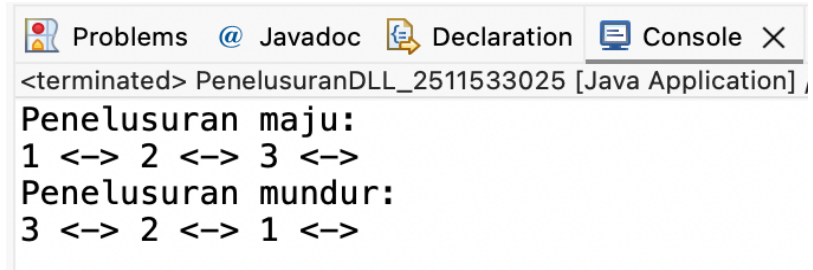


```

<terminated> InsertDLL_2511533025 [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plug
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

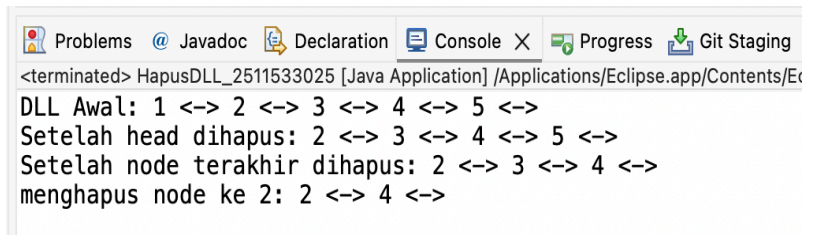
```

2. Output dari Class NodeDLL_2511533025 dan Class PenelusuranDLL_2511533025.



```
<terminated> PenelusuranDLL_2511533025 [Java Application],
Penelusuran maju:
1 <--> 2 <--> 3 <-->
Penelusuran mundur:
3 <--> 2 <--> 1 <-->
```

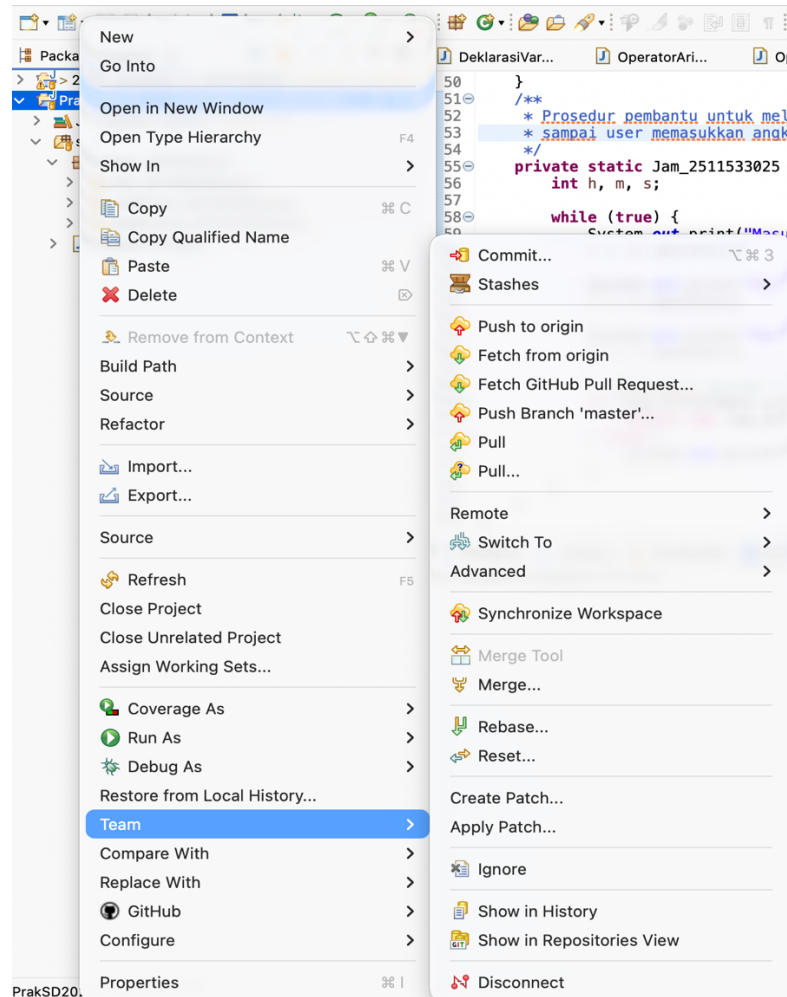
3. Output dari Class NodeDLL_2511533025 dan Class HapusDLL_2511533025.



```
<terminated> HapusDLL_2511533025 [Java Application] /Applications/Eclipse.app/Contents/E
DLL Awal: 1 <--> 2 <--> 3 <--> 4 <--> 5 <-->
Setelah head dihapus: 2 <--> 3 <--> 4 <--> 5 <-->
Setelah node terakhir dihapus: 2 <--> 3 <--> 4 <-->
menghapus node ke 2: 2 <--> 4 <-->
```

B. Langkah Penyimpanan

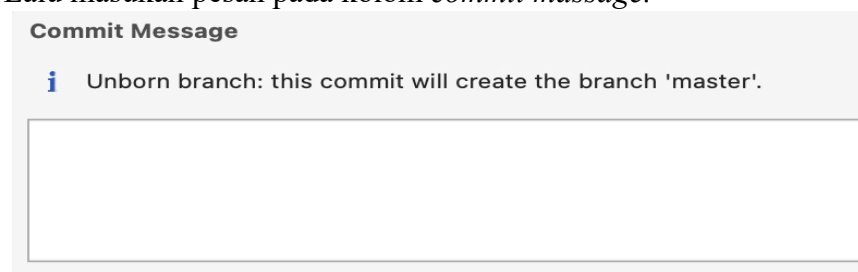
1. Sebelum kita masuk ke langkah, kita harus buat akun github dulu. Jika sudah buat akun github, baru bisa kita simpan di githubnya. Selanjutnya ikuti langkah ini. Tekan kanan mouse pada folder proyek, setelah itu pilih team, terus ke *commit*.



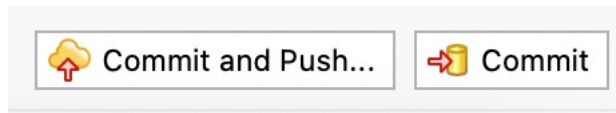
2. Lalu tekan tombol plus 2/double plus warna hijau jika ingin push semua kode, kalau hanya salah satu klik kode yang ingin di push, terus klik tanda plus single.



3. Lalu masukan pesan pada kolom *commit message*.



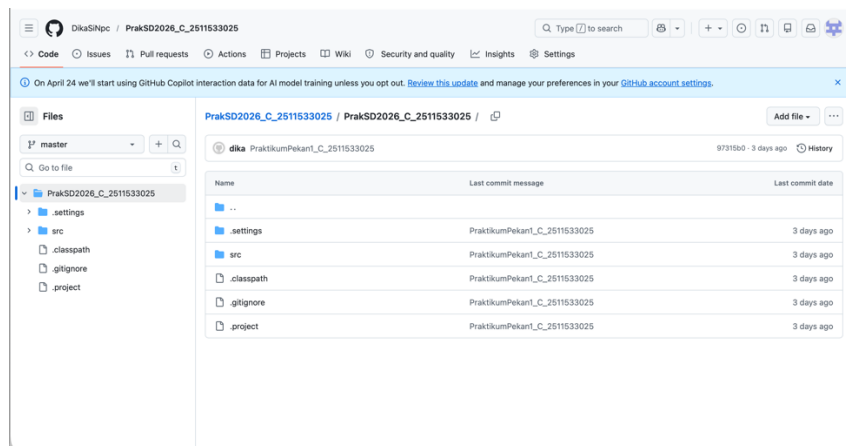
4. Setelah itu tekan *commit* and pus, lalu ikutin arahan untuk memasukan nama dan pw akun git hub kalian, maka projek kalian sudah tersimpan di akun github kalian.



5. Proyek yang sudah disimpan di akun proyek.

 DikaSiNpc/PrakSD2026_C_2511533025

6. Gambar proyek apabila sudah masuk di akun github.



Sedikit tambahan jika ingin cek hasil program nya tekan tombol *run* warna hijau.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru atau package yang baru.

BAB 3

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa implementasi *Double Linked List* menggunakan bahasa Java berhasil memberikan pemahaman mendalam mengenai struktur data linier yang memungkinkan penelusuran data secara dua arah. Setiap simpul (*node*) yang terdiri dari data, tautan ke simpul berikutnya (*next*), dan tautan ke simpul sebelumnya (*prev*) berhasil dimanipulasi melalui operasi dasar seperti penyisipan (*insert*), penghapusan (*delete*), pencarian (*search*), dan penelusuran (*traversal*) baik secara maju maupun mundur. Penggunaan referensi objek dalam Java terbukti efektif untuk menangani manajemen memori dan tautan antar node, meskipun memerlukan pengelolaan pointer yang lebih hati-hati dibandingkan *Single Linked List*. Secara keseluruhan, struktur data ini menunjukkan keunggulan dalam efisiensi operasi penghapusan dan navigasi data, menjadikannya pilihan yang tepat untuk aplikasi yang membutuhkan akses data bolak-balik secara cepat.

3.2 Saran

Untuk pengembangan lebih lanjut, disarankan agar mahasiswa tidak hanya berhenti pada implementasi dasar, tetapi juga mencoba mengembangkan variasi *Double Linked List* atau mengintegrasikannya dengan struktur data lain seperti *Stack* dan *Queue*. Selain itu, latihan mandiri dalam menangani *edge case* (seperti penghapusan pada daftar kosong atau satu elemen) serta optimalisasi penggunaan memori sangat diperlukan untuk meningkatkan ketangguhan kode. Disarankan pula untuk membandingkan kinerja *Double Linked List* dengan struktur data *array* dan *Single Linked List* dalam skenario penggunaan nyata guna memahami kapan struktur ini paling efektif diterapkan.

DAFTAR PUSTAKA

- Sumber daring (website):

[1] D. A. P. E., "Linked List: Single, Double, dan Circular Linked List," Universitas Telkom Jakarta, 13 Feb. 2026. [Online].

Tersedia: <https://jakarta.telkomuniversity.ac.id/linked-list-single-double-dan-circular-linked-list/>. [Diakses: 12 Mei 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 6

1. Intruksi Tugas Praktikum Pekan 6

Topik: Double Linked List

Tema: Playlist Lagu Sederhana

1. Deskripsi Tugas:

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

2. Aturan Penamaan (Wajib):

- Nama Kelas: pakai NIM lengkap. Contoh: Lagu_2511533025 dan Musik_2511533025
- Nama variabel & method: pakai 4 digit terakhir NIM. Contoh: judul_3025, tambahLagu_3025()
- Pointer: next_3025 dan prev_3025 juga wajib pakai akhiran NIM

3. Struktur yang Dibuat:

Kelas Lagu_NIM

Atribut yang harus ada:

- Judul (String)
- penyanyi (String)
- next (pointer ke Lagu)
- prev (pointer ke Lagu)

Method:

- constructor
- getter
- setter

Kelas Musik_NIM

Buat 5 operasi utama saja (nama method diakhiri 4 digit NIM):

- tambahLagu_3025() : menambah lagu baru di AKHIR playlist.
- hapusLaguAwal_3025() : menghapus lagu pertama (head).
- tampilMaju_3025() : menampilkan semua lagu dari awal ke akhir.
- tampilMundur_3025() : menampilkan semua lagu dari akhir ke awal (wajib DLL).
- cariLagu_3025(judul) : mencari lagu berdasarkan judul (tidak case-sensitive).

2. Kode program Class Lagu_2511533025

```

package TugasPekan6_2511533025;
class Lagu_2511533025 {
    private String judul_3025;
    private String penyanyi_3025;
    Lagu_2511533025 next_3025;
    Lagu_2511533025 prev_3025;
    // Constructor
    public Lagu_2511533025(String judul_3025, String
penyanyi_3025) {
        this.judul_3025 = judul_3025;
        this.penyanyi_3025 = penyanyi_3025;
        this.next_3025 = null;
        this.prev_3025 = null;
    }
    // Getter
    public String getJudul_3025() {
        return judul_3025;
    }
    public String getPenyanyi_3025() {
        return penyanyi_3025;
    }
    // Setter
    public void setJudul_3025(String judul_3025) {
        this.judul_3025 = judul_3025;
    }
    public void setPenyanyi_3025(String
penyanyi_3025) {
        this.penyanyi_3025 = penyanyi_3025;
    }
}

```

3. Kode program Class MusikDriver _2511533025

```

package TugasPekan6_2511533025;
import java.util.Scanner;
class MusikDriver_2511533025 {
    Lagu_2511533025 head_3025;
    Lagu_2511533025 tail_3025;
    // 1. Tambah Lagu
    public void tambahLagu_3025(String judul_3025,
String penyanyi_3025) {
        Lagu_2511533025 laguBaru_3025 =
            new Lagu_2511533025(judul_3025,
penyanyi_3025);
        if (head_3025 == null) {
            head_3025 = laguBaru_3025;
            tail_3025 = laguBaru_3025;
        } else {
            tail_3025.next_3025 = laguBaru_3025;
            laguBaru_3025.prev_3025 = tail_3025;
            tail_3025 = laguBaru_3025;
        }
    }
}

```

```

        System.out.println("Lagu berhasil
ditambahkan!");
    }
    // 2. Hapus Lagu Awal
    public void hapusLaguAwal_3025() {
        if (head_3025 == null) {
            System.out.println("Playlist kosong!");
        } else {
            System.out.println("Lagu dihapus: "
                + head_3025.getJudul_3025());
            head_3025 = head_3025.next_3025;
            if (head_3025 != null) {
                head_3025.prev_3025 = null;
            } else {
                tail_3025 = null;
            }
        }
    }
    // 3. Tampil Maju
    public void tampilMaju_3025() {
        if (head_3025 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511533025 bantu_3025 = head_3025;
        System.out.println("\n=== Playlist Maju ===");
        while (bantu_3025 != null) {
            System.out.println("Judul      : "
                + bantu_3025.getJudul_3025());
            System.out.println("Penyanyi: "
                + bantu_3025.getPenyanyi_3025());
            System.out.println();
            bantu_3025 = bantu_3025.next_3025;
        }
    }
    // 4. Tampil Mundur
    public void tampilMundur_3025() {
        if (tail_3025 == null) {
            System.out.println("Playlist kosong!");
            return;
        }
        Lagu_2511533025 bantu_3025 = tail_3025;
        System.out.println("\n=== Playlist Mundur
===");
        while (bantu_3025 != null) {
            System.out.println("Judul      : "
                + bantu_3025.getJudul_3025());
            System.out.println("Penyanyi: "
                + bantu_3025.getPenyanyi_3025());
            System.out.println();
            bantu_3025 = bantu_3025.prev_3025;
        }
    }
    // 5. Cari Lagu

```

```

        public void cariLagu_3025(String judulCari_3025) {
            Lagu_2511533025 bantu_3025 = head_3025;
            boolean ketemu_3025 = false;
            while (bantu_3025 != null) {
                if (bantu_3025.getJudul_3025()
                    .equalsIgnoreCase(judulCari_3025))
                {
                    System.out.println("\nLagu
ditemukan!");
                    System.out.println("Judul    : "
                        + bantu_3025.getJudul_3025());
                    System.out.println("Penyanyi: "
                        +
bantu_3025.getPenyanyi_3025());
                    ketemu_3025 = true;
                    break;
                }
                bantu_3025 = bantu_3025.next_3025;
            }
            if (!ketemu_3025) {
                System.out.println("Lagu tidak
ditemukan!");
            }
        }
        // Main Program
        public static void main(String[] args) {
            Scanner input_3025 = new Scanner(System.in);
            MusikDriver_2511533025 playlist_3025 =
                new MusikDriver_2511533025();
            int pilihan_3025;
            do {
                System.out.println("\n=== Playlist Musik
NIM: 2511533025 ===");
                System.out.println("1. Tambah Lagu");
                System.out.println("2. Hapus Lagu
Pertama");
                System.out.println("3. Lihat Playlist
(Maju)");
                System.out.println("4. Lihat Playlist
(Mundur)");
                System.out.println("5. Cari Lagu");
                System.out.println("6. Keluar");
                System.out.print("Pilihan: ");
                pilihan_3025 = input_3025.nextInt();
                input_3025.nextLine();
                switch (pilihan_3025) {
                    case 1:
                        System.out.print("Judul: ");
                        String judul_3025 =
                            input_3025.nextLine();
                        System.out.print("Penyanyi: ");
                        String penyanyi_3025 =
                            input_3025.nextLine();
                        playlist_3025.tambahLagu_3025(

```

```

        judul_3025,
        penyanyi_3025
    );
    break;
case 2:

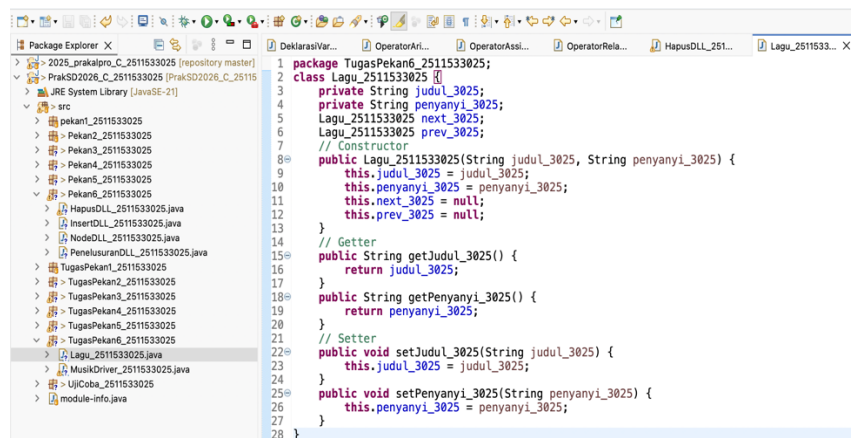
playlist_3025.hapusLaguAwal_3025();
    break;
case 3:
    playlist_3025.tampilMaju_3025();
    break;
case 4:
    playlist_3025.tampilMundur_3025();
    break;
case 5:
    System.out.print("Masukkan judul
lagu: ");
    String cari_3025 =
        input_3025.nextLine();

playlist_3025.cariLagu_3025(cari_3025);
    break;
case 6:
    System.out.println("Program
selesai.");
    break;
default:
    System.out.println("Pilihan tidak
valid!");
}
} while (pilihan_3025 != 6);
}
}

```

4. Gambar kode program dan output program.

1. Gambar Class Lagu_2511533025



2. Gambar Class MusikDriver_2511533025

```
DeklarasiVar... OperatorAri... OperatorAssi... OperatorRela... HapusDLL_251... Lagu_2511533...
1 package TugasPekan6_2511533025;
2 import java.util.Scanner;
3 class MusikDriver_2511533025 {
4     Lagu_2511533025 head_3025;
5     Lagu_2511533025 tail_3025;
6     // 1. Tambah Lagu
7     public void tambahLagu_3025(String judul_3025, String penyanyi_3025) {
8         Lagu_2511533025 laguBaru_3025 =
9             new Lagu_2511533025(judul_3025, penyanyi_3025);
10        if (head_3025 == null) {
11            head_3025 = laguBaru_3025;
12            tail_3025 = laguBaru_3025;
13        } else {
14            tail_3025.next_3025 = laguBaru_3025;
15            laguBaru_3025.prev_3025 = tail_3025;
16            tail_3025 = laguBaru_3025;
17        }
18        System.out.println("Lagu berhasil ditambahkan!");
19    }
20    // 2. Hapus Lagu Awal
21    public void hapusLaguAwal_3025() {
22        if (head_3025 == null) {
23            System.out.println("Playlist kosong!");
24        } else {
25            System.out.println("Lagu dihapus: "
26                + head_3025.getJudul_3025());
27            head_3025 = head_3025.next_3025;
28            if (head_3025 != null) {
29                head_3025.prev_3025 = null;
30            } else {
31                tail_3025 = null;
32            }
33        }
34    }
35    // 3. Tampil Maju
36    public void tampilMaju_3025() {
37        if (head_3025 == null) {
38            System.out.println("Playlist kosong!");
39            return;
40        }
41        Lagu_2511533025 bantu_3025 = head_3025;
42        System.out.println("\n=== Playlist Maju ===");
43        while (bantu_3025 != null) {
44            System.out.println("Judul : "
45                + bantu_3025.getJudul_3025());
46            System.out.println("Penyanyi: "
47                + bantu_3025.getPenyanyi_3025());
48            System.out.println();
49            bantu_3025 = bantu_3025.next_3025;
50        }
51    }
52    // 4. Tampil Mundur
53    public void tampilMundur_3025() {
54        if (tail_3025 == null) {
55            System.out.println("Playlist kosong!");
56            return;
57        }
58        Lagu_2511533025 bantu_3025 = tail_3025;
59        System.out.println("\n=== Playlist Mundur ===");
60        while (bantu_3025 != null) {
61            System.out.println("Judul : "
62                + bantu_3025.getJudul_3025());
63            System.out.println("Penyanyi: "
64                + bantu_3025.getPenyanyi_3025());
65            System.out.println();
66            bantu_3025 = bantu_3025.prev_3025;
67        }
68    }
}
```

```

69 // 5. Cari Lagu
70 public void cariLagu_3025(String judulCari_3025) {
71     Lagu_2511533025 bantu_3025 = head_3025;
72     boolean ketemu_3025 = false;
73     while (bantu_3025 != null) {
74         if (bantu_3025.getJudul_3025()
75             .equalsIgnoreCase(judulCari_3025)) {
76             System.out.println("\nLagu ditemukan!");
77             System.out.println("Judul : "
78                 + bantu_3025.getJudul_3025());
79             System.out.println("Penyanyi: "
80                 + bantu_3025.getPenyanyi_3025());
81             ketemu_3025 = true;
82             break;
83         }
84         bantu_3025 = bantu_3025.next_3025;
85     }
86     if (!ketemu_3025) {
87         System.out.println("Lagu tidak ditemukan!");
88     }
89 }
90 // Main Program
91 public static void main(String[] args) {
92     Scanner input_3025 = new Scanner(System.in);
93     MusikDriver_2511533025 playlist_3025 =
94         new MusikDriver_2511533025();
95     int pilihan_3025;
96     do {
97         System.out.println("\n=== Playlist Musik NIM: 2511533025 ===");
98         System.out.println("1. Tambah Lagu");
99         System.out.println("2. Hapus Lagu Pertama");
100        System.out.println("3. Lihat Playlist (Maju)");
101        System.out.println("4. Lihat Playlist (Mundur)");
102        System.out.println("5. Cari Lagu");
103        System.out.println("6. Keluar");
104        System.out.print("Pilihan: ");
105        pilihan_3025 = input_3025.nextInt();
106        input_3025.nextLine();
107        switch (pilihan_3025) {
108            case 1:
109                System.out.print("Judul: ");
110                String judul_3025 =
111                    input_3025.nextLine();
112                System.out.print("Penyanyi: ");
113                String penyanyi_3025 =
114                    input_3025.nextLine();
115                playlist_3025.tambahLagu_3025(
116                    judul_3025,
117                    penyanyi_3025
118                );
119                break;
120            case 2:
121                playlist_3025.hapusLaguAwal_3025();
122                break;
123            case 3:
124                playlist_3025.tampilMaju_3025();
125                break;
126            case 4:
127                playlist_3025.tampilMundur_3025();
128                break;
129            case 5:
130                System.out.print("Masukkan judul lagu: ");
131                String cari_3025 =
132                    input_3025.nextLine();
133                playlist_3025.cariLagu_3025(cari_3025);
134                break;
135            case 6:
136                System.out.println("Program selesai.");
137                break;
138            default:
139                System.out.println("Pilihan tidak valid!");
140        }
141    } while (pilihan_3025 != 6);
142 }
143 }

```

3. Gambar output kode program

```
Problems @ Javadoc Declaration Console X Progress Git Staging
<terminated> MusikDriver_2511533025 [Java Application] /Applications/Eclipse.app/Contents,

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Jiwa Yang Bersedih
Penyanyi: Ghea Indrawari
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Sanes
Penyanyi: Guyon Waton X Denny Caknan
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Merindukanmu
Penyanyi: Dash Uciha
Lagu berhasil ditambahkan!
```

=== Playlist Musik NIM: 2511533025 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 3

=== Playlist Maju ===

Judul : Jiwa Yang Bersedih
Penyanyi: Ghea Indrawari

Judul : Sanes
Penyanyi: Guyon Waton X Denny Caknan

Judul : Merindukanmu
Penyanyi: Dash Uciha

=== Playlist Musik NIM: 2511533025 ===

1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 4

=== Playlist Mundur ===

Judul : Merindukanmu
Penyanyi: Dash Uciha

Judul : Sanes
Penyanyi: Guyon Waton X Denny Caknan

Judul : Jiwa Yang Bersedih
Penyanyi: Ghea Indrawari


```

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Merindukanmu

Lagu ditemukan!
Judul   : Merindukanmu
Penyanyi: Dash Uciha

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu dihapus: Jiwa Yang Bersedih

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

=== Playlist Maju ===
Judul   : Sanes
Penyanyi: Guyon Waton X Denny Caknan

Judul   : Merindukanmu
Penyanyi: Dash Uciha

=== Playlist Musik NIM: 2511533025 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai.

```

5. Penjelasan sederhana tentang kode program

Program di atas merupakan implementasi sederhana dari struktur data Double Linked List dengan tema playlist lagu. Program terdiri dari

dua kelas, yaitu Lagu_2511533025 dan Musik_2511533025. Kelas Lagu_2511533025 digunakan untuk menyimpan data lagu seperti judul, penyanyi, serta pointer next_3025 dan prev_3025 yang menghubungkan lagu sebelumnya dan berikutnya. Kelas ini juga memiliki constructor, getter, dan setter.

Kelas Musik_2511533025 digunakan untuk mengelola playlist lagu. Method tambah Lagu_3025 () berfungsi menambahkan lagu di akhir playlist, sedangkan hapus LaguAwal_3025 () digunakan untuk menghapus lagu pertama. Method tampilMaju_3025 () menampilkan playlist dari awal ke akhir menggunakan pointer next, dan tampil Mundur_3025 () menampilkan playlist dari akhir ke awal menggunakan pointer prev. Selain itu, method cariLagu_3025 () digunakan untuk mencari lagu berdasarkan judul.

Pada method main, program menampilkan menu pilihan menggunakan perulangan do-while. Pengguna dapat menambah lagu, menghapus lagu, melihat playlist, mencari lagu, dan keluar dari program. Program ini menunjukkan cara kerja Double Linked List yang dapat diakses dari dua arah.